

# Midterm Report

Sean Gunn  
CMPSC 448

## Problem definition and Dataset curation (20 points)

---

### Problem:

As Large Language Models (LLMs) such as GPT, Claude, Gemini, Llama, and Qwen increasingly generate the text we read online, we face an emerging digital authorship crisis: Can we still tell who—or what—wrote a piece of text? Different LLMs may leave distinct linguistic, stylistic, and structural “fingerprints” in their responses, raising an intriguing question: Can machine learning identify the AI behind the text?

- **RQ1:** Can we identify which LLM generated a response from the output text alone?
- **RQ2:** Does the user's prompt help identify the LLM? (input only vs. output only vs. input + output)
- **RQ4:** What characteristics distinguish different LLMs? (which signals the classifier relies on, tested by removing them one at a time)

### Data curation:

Source: LMSYS Chatbot Arena 55k (lmarena-ai/arena-human-preference-55k), a public dataset on Hugging Face: <https://huggingface.co/datasets/lmarena-ai/arena-human-preference-55k>

How it was built (src/data/build\_dataset.py):

- Each Arena row (one prompt, two models' responses) is split into two examples: (llm\_name, llm\_input, llm\_output).
- Only the first turn of each conversation is used.
- Model versions are grouped into families: GPT, Claude, Gemini. Other models are dropped.
- Classes are balanced to 2,500 examples each (7,500 total, seed 42).
- Train/val/test is split by prompt (5,230 / 737 / 1,533), so the same prompt never appears in both train and test.

## Your CNN/RNN implementation and how you train it (i.e., your model details and training details) (20 points)

---

### CNN:

128-dimensional learned word embeddings feed three parallel one-dimensional convolutions with window sizes 3, 4, and 5 (100 filters each). Each convolution is followed by ReLU and

max-over-time pooling, giving a 300-dimensional vector, which passes through dropout (0.5) and a linear layer to three class scores. The model has 2,715,059 parameters.

### **LSTM :**

128-dimensional learned word embeddings (with dropout 0.5) feed a single-layer bidirectional LSTM with 128 hidden units per direction. Padded positions are skipped using packed sequences. The final forward and backward hidden states are concatenated (256 dimensions), passed through dropout (0.5), and classified with a linear layer. The model has 2,825,219 parameters.

**Preprocessing.** Text is lowercased and split into words, punctuation marks, and newlines (newlines are kept because formatting may be part of a model's fingerprint). Outputs are truncated to 256 tokens. The vocabulary is built from the training split only: the 20,000 most frequent tokens appearing at least twice, plus padding and unknown tokens (20,002 total).

**Training.** Both models use cross-entropy loss, the Adam optimizer (learning rate 0.001), and batch size 64, trained on CPU. Early stopping on validation accuracy picks the epoch to evaluate, and the test set is used once at the end with that best model. The CNN was trained for up to 15 epochs with early-stopping patience 3. The LSTM was trained for up to 25 epochs with patience 8, because it learns more slowly and its validation accuracy fluctuates more. The training seed was 0 (single run); the dataset sampling seed was 42.

## **The results and how they are presented (20 points)**

---

### **RQ1: Test results (output only)**

| Model | Test accuracy | Macro-F1 | Best val accuracy |
|-------|---------------|----------|-------------------|
| CNN   | 79.8%         | 0.800    | 80.1%             |
| LSTM  | 76.5%         | 0.767    | 77.7%             |

### **Per-family recall on the test set.**

| Family | CNN recall      | LSTM recall     |
|--------|-----------------|-----------------|
| Claude | 84.8% (441/520) | 82.3% (428/520) |
| GPT    | 80.4% (423/526) | 76.6% (403/526) |
| Gemini | 73.9% (360/487) | 70.2% (342/487) |

### **Confusion matrices:**

#### **CNN:**

| True/Predicted | Claude | GPT | Gemini |
|----------------|--------|-----|--------|
| Claude         | 441    | 49  | 30     |
| GPT            | 12     | 423 | 91     |
| Gemini         | 7      | 120 | 360    |

### LSTM:

| True/Predicted | Claude | GPT | Gemini |
|----------------|--------|-----|--------|
| Claude         | 428    | 56  | 36     |
| GPT            | 19     | 403 | 104    |
| Gemini         | 13     | 132 | 342    |

### Observations:

- Both models perform far above chance (about 80% vs. 33%), so responses from these three LLM families carry detectable fingerprints.
- The CNN (79.8%) scored 3.3 points higher than the LSTM (76.5%). With a single training seed per model, this is only a slight edge for the CNN, not a firm conclusion that the architecture is better
- Claude is the easiest family to identify. When the CNN predicts Claude it is correct 96% of the time (441 of 460 predictions), and the LSTM 93% (428 of 460).
- Most errors are between GPT and Gemini: the CNN labelled 91 GPT responses as Gemini and 120 Gemini responses as GPT (104 and 132 for the LSTM). The same error pattern in two different architectures suggests it reflects the data (similar response styles) rather than a quirk of one model.

### RQ2: Does the prompt help?

I compared three inputs for each model: the prompt only, the output only, and the prompt followed by the output (separated by a special token). The prompt was truncated to 256 tokens in the input-only setting and to 64 tokens when combined with the output.

### Test accuracy by input.

| Input               | CNN   | LSTM  |
|---------------------|-------|-------|
| Input (prompt) only | 35.4% | 34.4% |
| Output only         | 79.8% | 76.5% |
| Input + output      | 80.6% | 65.4% |

### Observations:

- The prompt alone does not identify the LLM. Both models score about 35%, close to the 33% expected from random guessing. In both input-only runs, training accuracy rose (CNN: 33% to 65%) while validation accuracy stayed flat at 33–37%. The models memorized the training prompts, but nothing carried over to unseen prompts.
- Adding the prompt to the output did not help the CNN. 80.6% with both vs. 79.8% with the output alone, a difference within reasonable noise.
- Adding the prompt hurt the LSTM. 65.4% with both vs. 76.5% with the output alone, an 11-point drop that is beyond noise. With the prompt in front, the LSTM's loss stayed near 1.0 (pure guessing) for the first five epochs before it started to learn. A likely explanation is that the 64 prompt tokens carry no information about the label and come before the informative part.
- Why input-only is near chance. The prompt carries almost no information about the label because the kind of question asked is nearly the same for all three families.
- Training settings matter for the LSTM. In my first run with the prompt added, early stopping (patience 3) ended training at epoch 5, just before the model began to learn, and the LSTM scored only 48.3%. With patience 8 it reached 65.4%.

### RQ4: What characteristics distinguish different LLMs? (ablations)

**Test accuracy and macro-F1 by ablation (output only; change in points vs. each model's baseline in parentheses).**

| <b>Ablation</b>                       | <b>CNN accuracy</b> | <b>CNN macro-F1</b> | <b>LSTM accuracy</b> | <b>LSTM macro-F1</b> |
|---------------------------------------|---------------------|---------------------|----------------------|----------------------|
| <b>No ablation (baseline)</b>         | <b>79.8%</b>        | <b>0.800</b>        | <b>76.5%</b>         | <b>0.767</b>         |
| No line breaks                        | 79.0% (−0.8)        | 0.792               | not run              | not run              |
| No markdown symbols (and line breaks) | 75.9% (−3.9)        | 0.761               | 73.1% (−3.4)         | 0.730                |
| No punctuation (and line breaks)      | 73.0% (−6.8)        | 0.729               | not run              | not run              |
| Shuffled word order                   | 68.3% (−11.5)       | 0.682               | 62.7% (−13.8)        | 0.629                |
| Opening 10 tokens removed             | 71.8% (−8.0)        | 0.720               | 67.1% (−9.4)         | 0.669                |
| Only first 32 tokens                  | 74.2% (−5.6)        | 0.742               | not run              | not run              |
| Content words only                    | 58.9% (−20.9)       | 0.589               | not run              | not run              |
| Function words only (content masked)  | 81.3% (+1.5)        | 0.813               | 82.3% (+5.8)         | 0.823                |

**Per-family recall by ablation (CNN, test set).**

| <b>Ablation</b>                       | <b>Claude recall</b> | <b>GPT recall</b> | <b>Gemini recall</b> |
|---------------------------------------|----------------------|-------------------|----------------------|
| <b>No ablation (baseline)</b>         | <b>84.8%</b>         | <b>80.4%</b>      | <b>73.9%</b>         |
| No line breaks                        | 83.3%                | 74.0%             | 79.9%                |
| No markdown symbols (and line breaks) | 82.9%                | 73.8%             | 70.6%                |
| No punctuation (and line breaks)      | 84.8%                | 70.3%             | 63.2%                |
| Shuffled word order                   | 73.7%                | 58.0%             | 73.7%                |
| Opening 10 tokens removed             | 74.2%                | 61.4%             | 80.3%                |
| Only first 32 tokens                  | 81.7%                | 67.1%             | 73.7%                |
| Content words only                    | 62.1%                | 48.7%             | 66.5%                |
| Function words only (content masked)  | 88.8%                | 78.7%             | 76.0%                |

**Per-family recall by ablation (LSTM, test set).**

| <b>Ablation</b>                       | <b>Claude recall</b> | <b>GPT recall</b> | <b>Gemini recall</b> |
|---------------------------------------|----------------------|-------------------|----------------------|
| <b>No ablation (baseline)</b>         | <b>82.3%</b>         | <b>76.6%</b>      | <b>70.2%</b>         |
| No markdown symbols (and line breaks) | 83.8%                | 71.3%             | 63.7%                |
| Shuffled word order                   | 59.2%                | 68.6%             | 60.0%                |
| Opening 10 tokens removed             | 76.3%                | 53.2%             | 72.3%                |
| Function words only (content masked)  | 89.8%                | 75.3%             | 81.7%                |

**Observations:**

- Most of the fingerprint is style, not topic. With all content words hidden, the CNN scored 81.3%, as good as the full text (79.8%). With only content words, it fell to 58.9%. The models are recognized more by how they write than by what they write about.
- (CNN) Line breaks don't matter, but punctuation and markdown symbols do. Removing line breaks changed accuracy by only 0.8 points. Removing all punctuation cost 6.8 points, and most of that loss fell on GPT (80.4% → 70.3%) and Gemini (73.9% → 63.2%). Claude stayed at 84.8%.
- The LSTM and the CNN agree. Both lose about 3–4 points without markdown symbols. Both lose 8–9 points without the opening tokens. Shuffling hurts both most: −11.5 for the CNN and −13.8 for the LSTM. So the signals are in the data, not a quirk of one architecture.
- Hiding content words did not hurt either model. The LSTM actually scored 82.3% with only function words and punctuation, above its own baseline (76.5%) and above the

CNN's 81.3%. The likely reason is that the vocabulary shrinks from 20,002 to 401 tokens, so the model has fewer rare words to memorize.

- Word order matters more for the LSTM. It reads the text in sequence, so shuffling hurts it the most.

## **Any in-depth analyses/experiments (20 points)**

---

LSTM performance was sensitive to the training settings. Results depended on how long I allowed training. With the input + output setting, early stopping with patience 3 ended training at epoch 5 (48.3% test accuracy), just before the loss began to fall. Allowing up to 25 epochs with patience 8 gave 65.4%. For output-only, the longer setting changed the result from 78.2% to 76.5%, a change within run-to-run noise, since validation accuracy fluctuates by several points between epochs.

In both architectures most mistakes are between GPT and Gemini (CNN: 91 GPT→Gemini and 120 Gemini→GPT; LSTM: 104 and 132), while Claude is rarely confused with the others. Two different models failing in the same places suggests the cause lies in the data. Data differences in response length and formatting probably is the cause of this.

In input-only runs, training accuracy rose to 65% (CNN) while validation accuracy stayed at 33–37%. The models memorize which prompts went with which label but nothing transfers to new prompts.

### **RQ4 analysis:**

To see what the models might be using, I measured 30 hand-made features of each response (length, lexical diversity, punctuation, formatting) and compared the three families. Gemini used far more markdown: 5.5 bold markers per response, against 1.6 for GPT and 0.02 for Claude. It also used more bullet lines (4.1, against 0.8 for GPT and 2.8 for Claude). Claude started 41% of its responses with “Here” (1,031 of 2,500), against 8% for GPT and 2% for Gemini. It also used “I” and apologies more often. GPT wrote the longest lines (34 words, against 25 for Claude and 21 for Gemini) and used “you” most.

The ablation experiments test whether the CNN and LSTM actually rely on these signals. Removing line breaks changed the CNN by only 0.8 points. Removing markdown symbols cost 3.9 points (LSTM: 3.4), and removing all punctuation cost 6.8 (CNN). Removing the first 10 tokens cost 8.0 points (LSTM: 9.4), and shuffling the word order cost 11.5 (LSTM: 13.8). Replacing every content word with <unk> did not hurt either model (CNN 81.3%, LSTM 82.3%), while keeping only content words dropped the CNN to 58.9%. So the fingerprints are mostly in how the models write and not in what they write about. The CNN and LSTM gave the same pattern, which suggests the cause is in the data and not in one architecture. These results come from single runs.

## **Lessons & Experience you learned in this project (20 points)**

---

The most important lesson was that data handling matters as much as modeling. The raw dataset was heavily skewed toward GPT, so I balanced it by sampling equally per family, and split by prompt rather than by row. Without the prompt-level split, the same prompt would appear in both train and test, and the input-only experiment would have looked far better than it really is. I also learned that a single number from a single run is not reliable. Validation accuracy fluctuated by several points between epochs, so a 1–2 point difference between the CNN and LSTM can be noise.

Training details turned out to matter. My first LSTM run on the prompt + output input scored 48.3% because early stopping with a small patience ended training just before the model started learning. With a longer allowance it reached 65.4%. Comparing the architectures, the CNN trained faster and more smoothly, and it was unaffected by adding irrelevant prompt text, while the LSTM was slower and harder to optimize. Finally, the results showed that LLM responses carry detectable fingerprints (about 80% accuracy on three classes) but that the prompt carries almost none.

Additionally, accuracy alone doesn't explain a model. Getting 80% showed that the families can be told apart, but only the ablations and feature analysis showed why. I would not have guessed that hiding every content word leaves accuracy unchanged.